hashlock.

Security Audit

Bad Development (DAO)

January 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

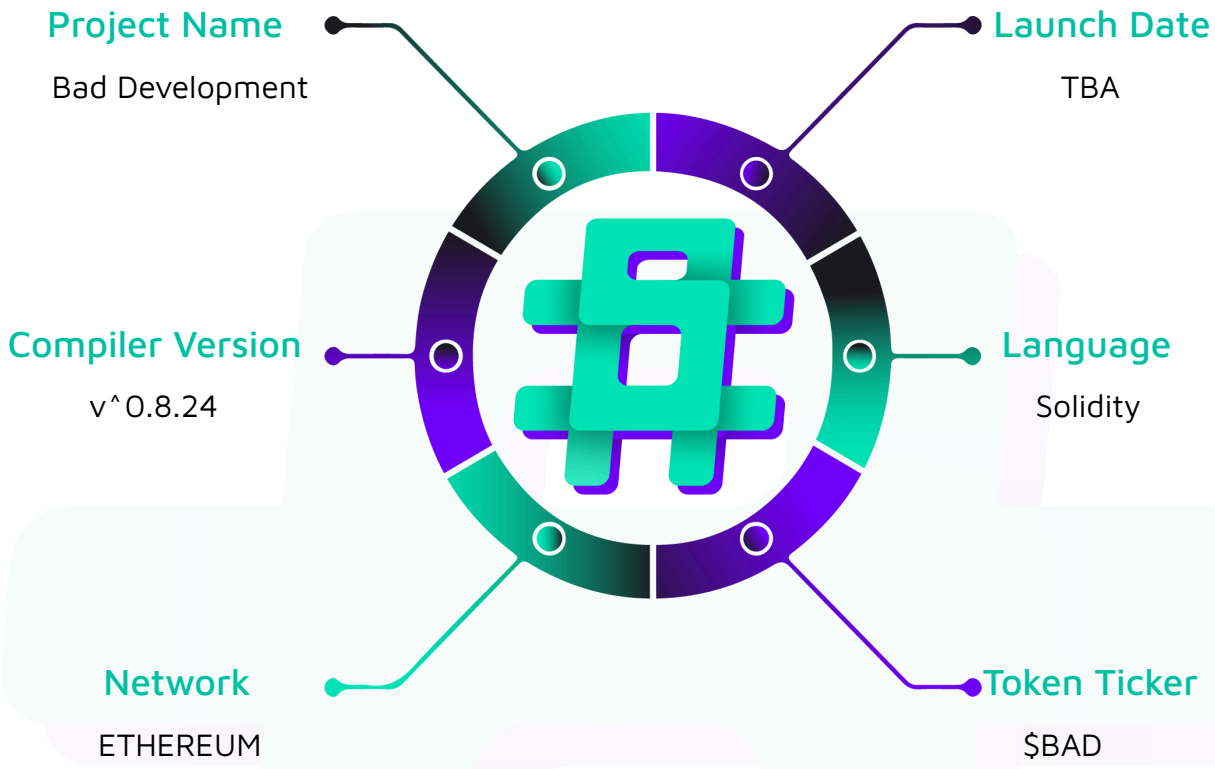
The Bad Development team partnered with Hashlock to conduct a security audit of their Token.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

BAD Development is revolutionizing the multimedia industry as a trailblazing first mover! At the heart of our mission are two groundbreaking features: Industry Philanthropy, where creators, artists, writers, and game developers benefit from weekly donations starting at $1,000 USD—decided by you!—and DAO/Direct Democracy Functions, empowering our community to make key decisions, from hiring talent to shaping creative projects. Join us on this transformative journey by reading our white paper and tuning into live streams to learn how you can be part of redefining the future of multimedia!

Project Name: Bad Development
Compiler Version: ^0.8.24
Website: https://baddevelopment.io/
Logo:

Visualised Context:



Project Visuals:

Audit scope

We at Hashlock audited the solidity code within the Bad Development project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Bad Development Protocol Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**January, 2025**
Contract 1	Token.sol
Contract 1 MD5 Hash	e11b913c9b2636b848de993578cfd3ca
GitHub Commit Hash	39cee20fc2f32460ff6de50a8e28f7a6af017d3b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

3 Medium severity vulnerabilities

4 Low severity vulnerabilities

4 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

BadDevelopment.sol
- ERC20 token with 9 decimals
- Allows admins to:
  - Enable/disable fees
  - Modify fee percentages and distribution
  - Set max wallet size
  - Set max transaction amount
  - Add/remove addresses from blacklist
  - Add/remove addresses from fee exemption list
  - Add/remove addresses from size limit exemption list
  - Add/remove liquidity pool addresses
  - Set revenue and LP token recipients
  - Set swap threshold for auto-liquidity
  - Recover stuck tokens/ETH from contract
  - Enable token after launch
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Bad Development project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Bad Development project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

High

[H-01] Token#getExceedsSizeLimit - The maximum wallet size limit is not correctly enforced which will result in users exceeding the limit

Description

The `getExceedsSizeLimit` function checks if a token transfer should fail. This is done by checking if the receiver address will or will not exceed the maximum wallet size if the inputted amount is added to their existing balance. However, this function also checks if the sender or the receiver is exempt from limits and if any are exempt, moves on with the transfer which makes these checks insufficient.

Vulnerability Details

The `getExceedsSizeLimit` function checks if a token transfer should fail.

```
   function getExceedsSizeLimits(address from, address to, uint256 amount) internal view returns (bool status) {
       if (isLimitExempt[from] || isLimitExempt[to]) {
            return false;
        }
                if (amount > maximumTransactionSize || (balanceOf(to) + amount) > maximumWalletSize) {
            return true;
        }
    }
```

As observed from the highlighted part, if the sender or the receiver is exempt from limits, the transaction will not revert due to size limits. However, this is problematic as the sender can be a limit exempt address such as the router, owner, or the liquidity pool while the receiver is not limit exempt. In this situation, the receiver address can exceed the maximum wallet size limit, breaking this invariant.

Impact

Any user can exceed wallet size limits.

Recommendation

Fix the function logic in order to sufficiently check for wallet size limits.

Status

Resolved

Medium

[M-01] Token#setFees - No upper limit on fees causes a critical centralization risk

Description

The `setFees` allows the contract owner to set the buy and sell fees. However, a lack of checks allows the owner to set these fees as high as 100% of the transaction's value.

Vulnerability Details

The `setFees` allows the contract owner to set the buy and sell fees.

```
function setFees(uint64 newBuyFee, uint64 newSellFee) external override onlyOwner {
    if (newBuyFee > DENOMINATOR) {
        revert InvalidBuyFee();
    } else if (newSellFee > DENOMINATOR) {
        revert InvalidSellFee();
    }

    buyFee = newBuyFee;
    sellFee = newSellFee;

    emit FeesChanged(newBuyFee, newSellFee);
```

```
    }
```

As observed in the function, the only checks done are to make sure that the fees do not exceed 100%. This means that the owner has unlimited control over the fees and is able to set it as high as 100%.
This creates a serious centralization risk as the owner can set really high fees and users might not expect these fees to be imposed upon their transactions, causing a serious loss of value to their transactions. The owner can even potentially front-run high-value transactions to apply high fees to these transactions.

Impact

This centralization risk can cause users to lose more tokens to fees than they have expected. Centralization risk also lowers the trust in the token.

Recommendation

Implement a maximum fee. An example is shown below.

```
    uint64 public constant MAX_FEE = 0.1e18; // 10%

    function setFees(uint64 newBuyFee, uint64 newSellFee) external override onlyOwner {
        if (newBuyFee > MAX_FEE) {
            revert InvalidBuyFee();
        } else if (newSellFee > MAX_FEE) {
            revert InvalidSellFee();
        }

        buyFee = newBuyFee;
        sellFee = newSellFee;

        emit FeesChanged(newBuyFee, newSellFee);
    }
```

Status

Resolved

[M-02] Token#constructor - Base liquidity pool is not limit exempt and it can not exceed the maximum wallet size

Description

BAD token imposes a maximum wallet size limit on its users except for select EOAs and important contracts. However, the base liquidity pool is not exempt from this limit in the constructor which can disrupt swaps. This address can be set as exempt from this limit but this can be overlooked which will impact the tokens trading.

Vulnerability Details

The `getExceedsSizeLimit` function is used in the `_update` function to see if the result of the transaction will cause the receiving address to exceed the maximum wallet size limit.

```
   function getExceedsSizeLimits(address from, address to, uint256 amount) internal view returns (bool status) {
        if (isLimitExempt[from] || isLimitExempt[to]) {
            return false;
        }
                if (amount > maximumTransactionSize || (balanceOf(to) + amount) > maximumWalletSize) {
            return true;
        }
    }
```

Take a look at the `constructor`.

```
   constructor(
        address _initialOwner,
        string memory _name,
        string memory _symbol,
        uint256 _maxSupply,
        address payable _revenueRecipient,
        address _lpTokenRecipient,
        address _uniswapV2Router02
```

```
    ) ERC20(_name, _symbol) ERC20Permit(_name) Ownable(_initialOwner) {
        /// Set transaction limits
        uint256 maxSupply = _maxSupply * 10 ** decimals();
        uint256 onePercentOfTotalSupply = intoUint256(ud(maxSupply).mul(ud(0.01e18)));
        maximumTransactionSize = onePercentOfTotalSupply;
        maximumWalletSize = onePercentOfTotalSupply;

        isLimitExempt[address(this)] = true;
        isLimitExempt[address(0)] = true;
        isLimitExempt[_initialOwner] = true;
        isLimitExempt[_uniswapV2Router02] = true;
        // rest of the function
```

It is observed that while some addresses are kept exempt from the limit, the base liquidity pool is not. To make sure no issues arise, the base liquidity pool should be kept exempt from the limit.

Impact

The base liquidity pool will not be able to exceed 1% of the total supply limit put on any address. This will seriously affect the trading of the token.

Recommendation

Set the `basePair` as `isLimitExempt`.

Status

Resolved

[M-03] Token#_update - Maximum wallet size limit checks are done with the amount that is not fee reduced which causes reverts when it shouldn't

Description

BAD token imposes a maximum wallet size limit on its users. Any transaction is checked if the receiving address exceeds this limit as a result of this transaction. However, the amount that is used to check if the user will exceed this limit is not the actual amount

that the user will receive after fees. This causes the transactions to revert when they shouldn't.

Vulnerability Details

The `getExceedsSizeLimit` function is used in the `_update` function to see if the result of the transaction will cause the receiving address to exceed the maximum wallet size limit.

```
   function getExceedsSizeLimits(address from, address to, uint256 amount) internal view
returns (bool status) {
        if (isLimitExempt[from] || isLimitExempt[to]) {
            return false;
        }
                if (amount > maximumTransactionSize || (balanceOf(to) + amount) >
maximumWalletSize) {
            return true;
        }
    }
```

Take a look at the `_update` function.

```
   function _update(address from, address to, uint256 value) internal virtual override {
        // Rest of the function
        } else {
            if (getExceedsSizeLimits(from, to, value)) {
                revert ExceedsSizeLimit();
            }

            if (shouldTakeFee(from, to)) {
                uint256 transferAmount = value;
                uint256 feeAmount = getFeeAmount(to, transferAmount);
                transferAmount -= feeAmount;

                super._update(from, address(this), feeAmount);
                // Rest of the function
                // Send on to caller
```

```
            super._update(from, to, transferAmount);
```

It is observed that the size limits are checked with the inputted `value` while the user wallet receives `value - feeAmount`. This means that even if the user does not exceed the wallet size after fees are subtracted from the transfer amount, the transaction will still fail.

In an example scenario where the user is 99 tokens short of the maximum wallet size.

1) The user gets transferred 100 tokens
2) With 5% fees, the user will receive 95 tokens
3) The transaction will still fail as the maximum wallet size check is done with the user balance + 100 instead of the actual amount the user receives.

Impact

Transactions that should not fail, will fail as the contract incorrectly calculates the user's token amount after the transaction.

Recommendation

Correctly check if the user exceeds the maximum wallet size by using the amount that is subtracted with fee amounts if fees are applicable to this transaction.

Status

Resolved

Low

[L-01] Token - Prevent setting a state variable with the same value

Description

Not only is wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario. Instances are shown below.

```
    function setTransferFeeStatus()
    function setTradeFeeStatus()
    function setFees()
    function setFeeSplit()
    function setLiquidityPool()
    function setRevenueRecipient()
    function setLpTokenRecipient()
    function setSwapThreshold()
    function setSizeLimits()
    function setFeeExemption()
    function setLimitExemption()
```

Recommendation

Implement checks in your functions to avoid setting state variables to their existing values. Prior to updating a state variable, compare the new value with the current value and proceed with the assignment only if they differ. An example is shown below.

```
   function setTransferFeeStatus(bool status) external override onlyOwner {
        if (status == isTransferFeeEnabled) {
            revert TransferFeeStatusUnchanged();
        }
        isTransferFeeEnabled = status;
        emit TransferFeeStatusChange(status);
    }
```

Status

Resolved

[L-02] Token#constructor - Missing checks for `address(0)` in constructor

Description

It is good practice to implement checks for `address(0)` to prevent accidental inputs that can lead to the need for redeploying contracts or extra transactions to fix. Instances are highlighted below.

```
    constructor(
        address _initialOwner,
        string memory _name,
        string memory _symbol,
        uint256 _maxSupply,
        address payable _revenueRecipient,
        address _lpTokenRecipient,
        address _uniswapV2Router02
    ) ERC20(_name, _symbol) ERC20Permit(_name) Ownable(_initialOwner) {
```

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding required statements that ensure address parameters are not the zero address. An example is shown below.

```
    constructor(...) {
        require(_revenueRecipient != address(0) && _lpTokenRecipient != address(0), "Invalid address");
    }
```

Status

Resolved

[L-03] Token - Centralization risk for privileged roles and rug pull risk

Description

The contract has functions of critical importance that can seriously impact the functionality of the token. These functions are only accessible by the owner role which means that the owner address needs to be trusted. This introduces a single point of failure where the owner can act maliciously or introduce accidental updates.

Additionally, all the tokens being minted to a single address introduces a rug pull risk.

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders. This approach enhances security, reduces the risk of malicious actions by a single entity, and prevents single points of failure.

Note

The BAD Development team has been informed about best security practices about multi-sig solutions for contract ownership.

Status

Acknowledged

[L-04] Token - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Status

Resolved

Gas

[G-01] Token#addToBlacklist - Array length should be cached outside the loop

Description

Caching the array length outside a loop saves reading it on each iteration, as long as the array's length is not changed during the loop.

Recommendation

Cache the array length by assigning it to a variable in memory before looping through the array. An example is shown below.

```
   function addToBlacklist(address[] memory accounts) external override onlyOwner {
        uint256 length = accounts.length;
        for (uint256 i = 0; i < length; ++i) {
            isBlacklisted[accounts[i]] = true;
        }
        emit AccountsBlacklisted(accounts);
    }
```

Status

Resolved

[G-02] Token#_update, setFeeSplit - Calculation can be unchecked

Description

The `_update` function calculates the `transferAmount` by subtracting the `feeAmount` from the `value`. This subtraction can be unchecked since the `feeAmount` can not be greater than the `transferAmount`.

The `setFeeSplit` function has the same optimization possible. Since the `revenueShare` value can not be greater than the `DENOMINATOR`, the subtraction can be made unchecked.

Recommendation

Uncheck the subtraction. An example is shown below.

```
    function _update(address from, address to, uint256 value) internal virtual override {
        //rest of the function
            if (shouldTakeFee(from, to)) {
                uint256 transferAmount = value;
                uint256 feeAmount = getFeeAmount(to, transferAmount);
                unchecked {
                    transferAmount -= feeAmount;
                }
        //rest of the function
```

Status

Resolved

[G-03] Token#setFeeSplit - Avoid repeating the same calculation

Description

The `setFeeSplit` function repeats the same calculation unnecessarily, causing a waste of gas.

```
    function setFeeSplit(uint64 revenueShare) external override onlyOwner {
        if (revenueShare > DENOMINATOR) {
            revert ShareExceedsDenominator();
```

```
        }

        liquidityPercentage = DENOMINATOR - revenueShare;
        revenuePercentage = revenueShare;

        emit FeeSplitChanged(DENOMINATOR - revenueShare, revenueShare);
    }
```

Recommendation

Avoid repeating the same calculation. An example is shown below.

```
    function setFeeSplit(uint64 revenueShare) external override onlyOwner {
        if (revenueShare > DENOMINATOR) {
            revert ShareExceedsDenominator();
        }

        liquidityPercentage = DENOMINATOR - revenueShare;
        revenuePercentage = revenueShare;

        emit FeeSplitChanged(liquidityPercentage, revenueShare);
    }
```

Status

Resolved

[G-04] Token#shouldTakeFee, _update, getIsTokenEnabled - Simpler condition should be applied first

Description

The `shouldTakeFee` function should check for the simple boolean check before moving on to the more complex check that involves comparisons.

The `_update` function should check for the `getSwapGuardState` condition before moving on to the more complex checks.

The `getIsTokenEnabled` function should check if the token is enabled and return true before moving on to more comparisons

Recommendation

Update the functions as shown below.

```
function shouldTakeFee(address from, address to) internal view returns (bool) {
    if (isFeeExempt[from] || isFeeExempt[to]) {
        return false;
    }
    if (isTransferFeeEnabled) {
        return true;
    }
    if (isTradeFeeEnabled && (isLiquidityPool[from] || isLiquidityPool[to])) {
        return true;
    }
    return false;
}
function _update(address from, address to, uint256 value) internal virtual override {
    if (getSwapGuardState()) {
        super._update(from, to, value);
    } else {
            // rest of the function

function getIsTokenEnabled(address from, address to) internal view returns (bool) {
    address owner = owner();
    bool isEnabled = isTokenEnabled;
    if (isEnabled) {
        return true;
    }
    //rest of the function
```

Status

Resolved

Centralisation

The Bad Development project values security and efficiency over decentralisation.
Bad Development emphasizes a security-focused approach while ensuring operational functionality. By centralizing critical functions, the project enhances reliability and rapid decision-making while maintaining accountability, ensuring the system remains robust and efficient.

Conclusion

After Hashlock's analysis, the Bad Development project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au